STUDENT STUDY PLANNER APPLICATION

MINI PROJECT REPORT

Submitted By:

Darwin J – 814725148009

Deenadhayalan S - 814725148010

Dharani M - 814725148011

Dharani R - 814725148012

BACHELOR OF ENGINEERING

IN

COMPUTER SCIENCE AND ENGINEERING (ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING)

SRM TRP ENGINEERING COLLEGE, TIRUCHIRAPPALLI

NOV / DEC – 2025

TABLE OF CONTENTS

1. Abstract

2. Introduction

3. Problem Definition

4. Objectives of the Study

5. Scope of the Project

6. Literature Review

7. System Analysis

- o 7.1 Core concepts
- o 7.2 Architecture
- o 7.3 Instruction diagram

8. System Design

- o 8.1 System Architecture
- o 8.2 Data Structure Design (Multi Linked List)
- o 8.3 Class Diagram / Pseudo Code Overview

9. Implementation Details

- o 9.1 Algorithm Explanation
- o 9.2 Code Structure (Conceptual Overview)

10.Working Mechanism of Multi Linked List in Library Tracking

11.Use Case Diagram and Explanation

12.Testing and Validation

13.Result and Discussion

14.Limitations

15.Conclusion

16.References

1. ABSTRACT

The Student Study Planner Application is a software solution designed to assist students in organizing and managing their academic activities effectively. In today's fast-paced educational environment, students often struggle to balance multiple subjects, assignments, and deadlines, leading to poor time management and decreased productivity. This application addresses these challenges by providing a structured platform to plan, schedule, and monitor study tasks efficiently.

The system allows users to add, update, and delete study tasks, set deadlines, and track their completion status. By using dynamic data structures such as linked lists, the application ensures flexible and efficient handling of task data. It enables students to maintain a clear overview of their pending and completed tasks, thereby improving their ability to prioritize work and meet deadlines.

Additionally, the application promotes disciplined study habits by encouraging consistent tracking of progress. The simple and user-friendly interface makes it easy for students to interact with the system without requiring advanced technical knowledge. Overall, the Student Study Planner Application demonstrates how programming concepts and data structures can be effectively applied to solve real-world problems and enhance student productivity.

2. INTRODUCTION

In the modern educational environment, students are required to manage multiple academic responsibilities such as attending classes, completing assignments, preparing for exams, and participating in extracurricular activities. Without proper planning, handling these tasks can become overwhelming, often resulting in missed deadlines, poor time management, and reduced academic performance. Therefore, an effective study planning system is essential to help students organize their work and utilize their time efficiently.

The Student Study Planner Application is designed to provide a simple and structured solution for managing study-related activities. It allows students to create and maintain a schedule of tasks, set deadlines, and monitor their progress. By organizing tasks in a systematic way, the application helps students prioritize their work and focus on important subjects.

This project demonstrates the practical application of programming concepts and data structures, particularly linked lists, in solving real-life problems. The system is developed with a user-friendly interface, making it accessible to students with basic computer knowledge. Overall, the application aims to improve productivity, reduce stress, and promote disciplined study habits among students.

3. PROBLEM DEFINITION

In academic life, students are required to manage multiple subjects, assignments, and deadlines simultaneously. However, many students face difficulties in organizing their study schedules effectively due to lack of proper planning tools. Traditional methods such as maintaining notes in notebooks or relying on memory are often unreliable and inefficient. These methods do not provide a structured way to track tasks, leading to missed deadlines, incomplete assignments, and increased stress.

One of the major problems is poor time management, where students are unable to allocate sufficient time for each subject. Additionally, there is no proper system to monitor task completion or track progress, which makes it difficult for students to evaluate their performance. Managing multiple tasks without prioritization further adds to confusion and inefficiency.

Existing digital tools may offer planning features, but they are often complex, require internet access, or are not specifically designed for student needs. Therefore, there is a need for a simple, efficient, and user-friendly system that helps students plan their study activities, manage tasks, and improve productivity. The Student Study Planner Application is designed to address these issues by providing a structured and effective solution for academic task management.

4. OBJECTIVES OF THE STUDY

The main objective of the Student Study Planner Application is to develop an efficient system that helps students organize and manage their academic activities in a structured manner. The application aims to improve productivity and ensure better utilization of time by providing a clear and systematic approach to study planning.

The specific objectives of this study are as follows:

- To design and develop a simple and user-friendly study planning application for students.
- To help students manage their study schedules effectively by organizing tasks and subjects.
- To provide a facility for adding, updating, and deleting study tasks dynamically.
- To enable students to set deadlines and track the completion status of tasks.
- To improve time management skills by prioritizing important tasks.
- To monitor student progress and provide a clear view of pending and completed work.
- To reduce stress and confusion caused by unorganized study routines.
- To demonstrate the practical implementation of data structures like linked lists in real-world applications.

Overall, the project aims to create a reliable and efficient system that enhances study habits and supports academic success.

5. SCOPE OF THE PROJECT

The Student Study Planner Application is designed to assist students in organizing and managing their academic tasks efficiently. The scope of this project is focused on providing a simple and structured system that helps students plan their daily and weekly study activities. It allows users to create, update, and manage study tasks along with their deadlines, ensuring better time management and task prioritization.

The system enables students to maintain a list of subjects and associated tasks, track their completion status, and view pending work in an organized manner. It provides functionalities such as adding new tasks, deleting completed or unnecessary tasks, and marking tasks as completed. These features help students maintain a clear overview of their study schedule and progress.

This project is mainly intended for academic purposes and demonstrates the use of data structures like linked lists for dynamic data management. It is suitable for small to moderate levels of task handling and can be used by individual students for personal study planning..

6. LITERATURE REVIEW

The development of the Student Study Planner Application is based on concepts derived from existing study methods, digital tools, and data structure techniques. A review of these approaches helps in understanding their advantages and limitations, which guided the design of this system.

1. Traditional Study Planning Methods

Traditionally, students use notebooks, diaries, or simple to-do lists to manage their study schedules. While these methods are easy to use, they lack proper organization, flexibility, and tracking capabilities. It becomes difficult to update tasks, prioritize subjects, and monitor progress effectively. These limitations often result in missed deadlines and inefficient time management.

2. Existing Digital Study Tools

Modern applications such as task managers and calendar-based tools provide digital solutions for planning. These tools offer features like reminders, scheduling, and notifications. However, many of them are complex, require continuous internet access, and are not specifically designed for academic purposes. Some applications also include unnecessary features that make them difficult for students to use effectively.

3. Use of Data Structures in Task Management

Data structures such as linked lists are widely used in applications that require dynamic data handling. Linked lists allow efficient insertion and deletion of elements without requiring memory allocation.

7. SYSTEM ANALYSIS

System analysis is an important phase in software development where the requirements, components, and overall structure of the system are studied. In the Student Study Planner Application, system analysis helps in identifying how tasks are managed, stored, and processed efficiently.

7.1 Core Components

The Student Study Planner Application consists of the following core components:

- **Task Manager:**
  Handles all study-related tasks such as adding, updating, deleting, and marking tasks as completed.
- **Schedule Planner:**
  Organizes tasks based on subjects and deadlines, helping students plan their study time effectively.
- **Progress Tracker:**
  Monitors completed and pending tasks, giving students a clear view of their study progress.
- **Data Structure (Linked List):**
  Stores tasks dynamically, allowing easy insertion and deletion without memory wastage.

7.2 Architecture

The system follows a simple and efficient architecture consisting of three main layers:

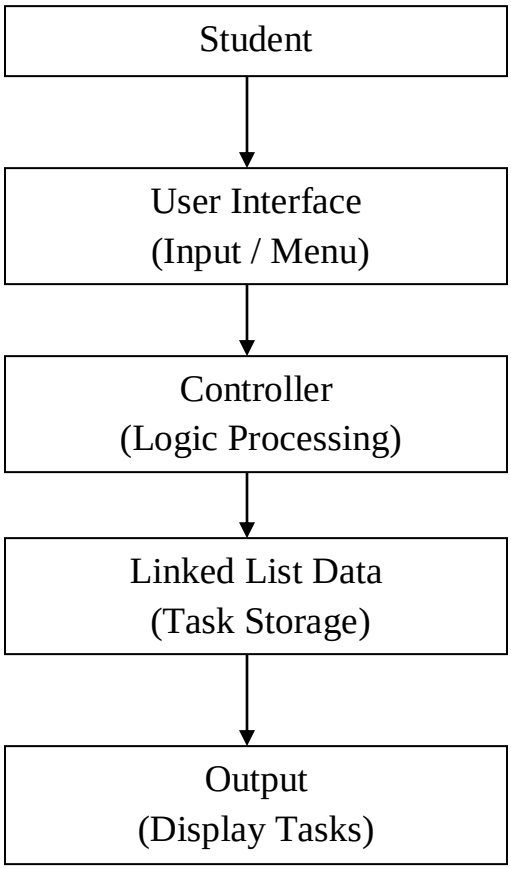
- **User Interface:**
  Accepts input from the user such as task details, choices, and commands.
- **Controller Module:**
  Processes user input and performs operations like adding tasks, searching, and updating status.
- **Data Layer:**
  Stores task data using a linked list and manages all data-related operations.

Flow of the System:

```
User Input → Controller → Linked List (Data Storage) → Output Display
```

This architecture ensures smooth interaction between the user and the system.

7.3 Infrastructure Diagram:



8. SYSTEM DESIGN

This section describes the overall design of the Student Study Planner Application, including its architecture, data structure, and logical representation. The system is designed to efficiently manage student tasks using a multi-linked list, enabling access through multiple perspectives such as subject, priority, and deadline.

8.1 System Architecture

This subsection explains the overall structure and working flow of the system.

Components:

- **User Interface (Console-based):**
  Accepts user input for operations like Add Task, View Tasks, Complete Task, and Delete Task.

- **Controller Module:**
  Processes user requests and performs logical operations such as insertion, deletion, searching, and updating tasks.
- **Data Layer:**
  Implements the multi-linked list to store and manage task records efficiently.

Architecture Flow:

User Input → Controller → Multi-Linked List → Output

Functional Modules:

- Add Task
- Display Tasks (by Subject, Priority, Deadline)
- Search Task
- Complete Task
- Delete Task

Traversal Paths:

- Subject-wise traversal
- Priority-wise traversal
- Deadline-wise traversal

This architecture ensures efficient task management and flexible data access.

8.2 Data Structure Design (Multi Linked List)

This subsection describes the core data structure used in the system.

A **multi-linked list** is used where each task node contains multiple pointers, allowing it to be part of different linked lists simultaneously.

Task Node Structure:

```
typedef struct Task {
    int id;
    char subject[50];
    char taskName[100];
    char deadline[20];
    int priority;   // High, Medium, Low
    int isCompleted;
```

```
    struct Task *nextBySubject;
    struct Task *nextByPriority;
    struct Task *nextByDeadline;

} Task;
```

Key Features:

- Each task is represented by a single node
- Multiple pointers connect the node to different lists
- Allows multiple views of the same data

Benefits:

- Efficient multi-dimensional traversal
- Reduces data redundancy
- Faster access based on different criteria
- Flexible and dynamic structure

8.3 Class Diagram / Pseudo Code Overview

Class Diagram (Conceptual View)

Task
- id - subject - taskName - deadline - priority - isCompleted - nextBySubject - nextByPriority - nextByDeadline

Pseudo Code

Add Task

```
function addTask(id, subject, taskName, deadline, priority):
    create new Task node
    insert into subject list
```

```
    insert into priority list
    insert into deadline list
```

Complete Task

```
function completeTask(id):
    task = searchById(id)
    if task != NULL:
        task.isCompleted = 1
        print "Task completed"
    else:
        print "Task not found"
```

Search Task

```
function searchTask(id):
    traverse list
    return task if found
```

Delete Task

```
function deleteTask(id):
    remove node from all linked lists
    free memory
```

Conclusion of System Design:

The system design of the Student Study Planner Application uses a multi-linked list to efficiently manage tasks across multiple dimensions. This approach improves performance, reduces redundancy, and provides flexible access to data. It demonstrates how advanced data structures can be applied to real-world student productivity systems.

9. IMPLEMENTATION DETAILS

This section explains how the Student Study Planner Application is implemented using a multi-linked list data structure. It describes the logic behind key operations and the overall organization of the program. The implementation focuses on efficient task management, allowing users to add, search, update, and delete tasks dynamically.

9.1 Algorithm Explanation

This subsection explains the step-by-step logic used to perform major operations in the system.

➢Add a New Task

- **Input:** Task ID, Subject, Task Name, Deadline, Priority
- **Steps:**

1. Create a new task node using dynamic memory allocation (malloc).
2. Assign values to all fields (id, subject, taskName, deadline, priority).
3. Initialize all pointers (nextBySubject, nextByPriority, nextByDeadline) to NULL.
4. Insert the node into:
    - Subject list (grouped by subject)
    - Priority list (sorted by priority level)
    - Deadline list (sorted by deadline)
5. Confirm task addition.

➢Search Task

- **Input:** Task ID or Task Name
- **Steps:**

1. Start traversal from the head node.
2. Compare each node's ID or task name.
3. If a match is found, display task details.
4. If not found, return "Task not found".

➢Complete Task

- **Input:** Task ID
- **Steps:**

1. Search for the task using ID.
2. Check if the task exists.
3. If found, update isCompleted = 1.
4. Display confirmation message.

➢Delete Task

- **Input:** Task ID
- **Steps:**

1. Traverse all three linked lists.

2. Locate the node in each list.
3. Remove (unlink) the node from:
    - Subject list
    - Priority list
    - Deadline list
4. Free the memory using free().
5. Confirm deletion.

➢Display Tasks

- **Input:** Choice (Subject / Priority / Deadline)
- **Steps:**

1. Select the corresponding list head.
2. Traverse using appropriate pointer.
3. Display task details in order.

9.2 Code Structure (Conceptual Overview)

This subsection describes how the program is organized into different modules for clarity and maintainability.

➢Modules

- addTask() – Adds a new task
- searchTask() – Searches for a task
- completeTask() – Marks task as completed
- deleteTask() – Deletes a task
- displayTasks() – Displays tasks based on criteria

➢Sample Code Snippet

```
Task* createTask(int id, char* subject, char* taskName, char* deadline, int priority) {
    Task* newTask = (Task*)malloc(sizeof(Task));

    newTask->id = id;
    strcpy(newTask->subject, subject);
    strcpy(newTask->taskName, taskName);
    strcpy(newTask->deadline, deadline);
    newTask->priority = priority;
    newTask->isCompleted = 0;
```

```
    newTask->nextBySubject = NULL;
    newTask->nextByPriority = NULL;
    newTask->nextByDeadline = NULL;

    return newTask;
}
```

10. WORKING MECHANISM OF MULTI-LINKED LIST IN STUDENT STUDY PLANNER APPLICATION

1. Concept Overview

A multi-linked list is a data structure in which each node contains multiple pointers, allowing it to be part of several linked lists simultaneously. In the Student Study Planner Application, this structure is used to manage study tasks efficiently by organizing them based on different attributes such as subject, priority, and deadline.

This approach eliminates the need for duplicate data and allows tasks to be accessed in multiple ways without redundancy.

2. Task Node Structure

Each task in the system is represented by a single node that contains:

- Task details:
  - Task ID
  - Subject
  - Task Name
  - Deadline
  - Priority
  - Completion Status
- Multiple pointers:
  - **nextBySubject** → links tasks belonging to the same subject
  - **nextByPriority** → links tasks based on priority level
  - **nextByDeadline** → links tasks sorted by deadline

This allows a single task node to be connected to multiple lists at the same time.

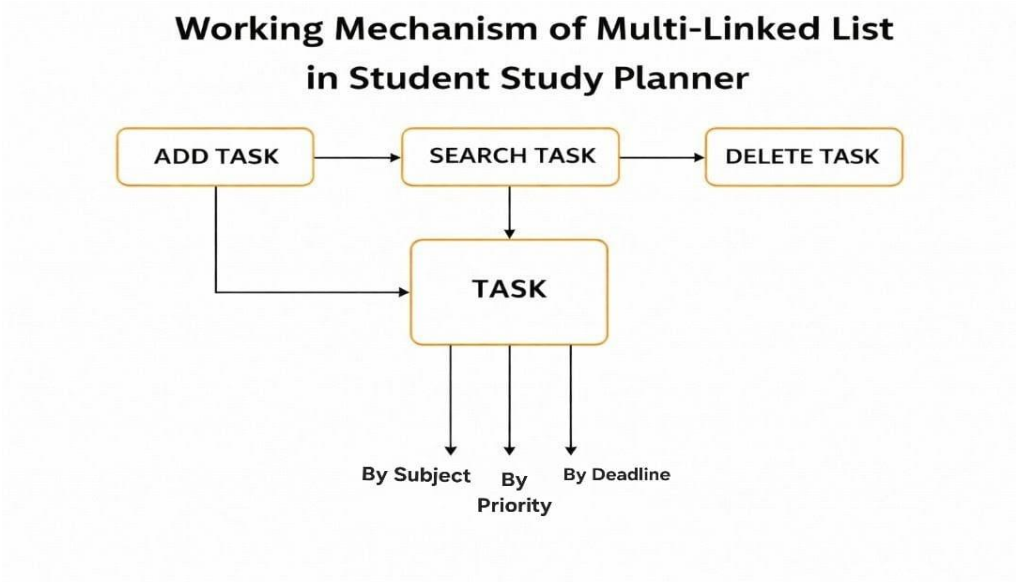
3. Working Mechanism

- When a new task is created, it is inserted into all relevant linked lists.

- The system maintains separate head pointers for each list:
    - Subject-wise list
    - Priority-wise list
    - Deadline-wise list
- Each list provides a different view of the same set of tasks:
    - Subject list helps in subject-based study planning
    - Priority list helps in focusing on important tasks
    - Deadline list helps in completing tasks on time
- When a task is updated (e.g., marked as completed), the change is reflected across all lists since they share the same node.
- When a task is deleted, it is removed from all linked lists to maintain consistency.

4. Advantages of This Mechanism

- **Efficient Data Access:** Tasks can be accessed using multiple criteria without duplication.
- **Reduced Redundancy:** Only one node is used for each task.
- **Flexible Traversal:** Allows viewing tasks in different ways (subject, priority, deadline).
- **Dynamic Updates:** Easy insertion, deletion, and modification of tasks.
- **Improved Performance:** Faster retrieval compared to single linked list systems.



11. USE CASE DIAGRAM AND EXPLANATION

A use case diagram represents the interaction between the user (actor) and the system functionalities. It provides a clear understanding of how the Student Study Planner Application operates from the user's perspective. The diagram shows the different actions a student can perform and how the system responds to those actions.

Diagram Description:

- The **actor** in this system is the *Student*, represented by a stick figure.
- The **use cases** are represented as ovals, which include the main functionalities of the system.
- All use cases are enclosed within a system boundary labeled **"Student Study Planner Application."**
- Lines are used to connect the actor to the use cases, indicating interaction.

Main Use Cases:

- **Add Task:**
  Allows the student to create a new study task by entering details such as subject, task name, deadline, and priority.
- **View Tasks:**
  Displays all tasks in an organized manner based on subject, priority, or deadline.
- **Search Task:**
  Enables the student to find a specific task using task ID or name.
- **Complete Task:**
  Allows the student to mark a task as completed once it is finished.
- **Delete Task:**
  Removes a task from the system when it is no longer required.
- **Track Progress:**
  Shows the number of completed and pending tasks, helping students evaluate their performance.

Relationships:

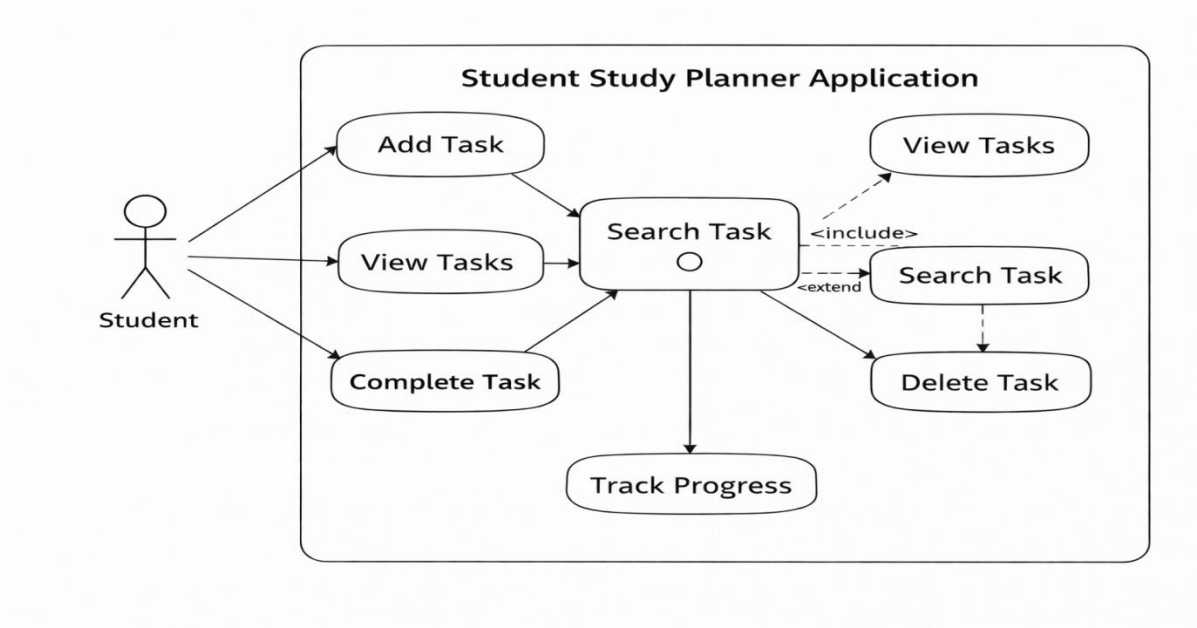
- **Include Relationship:**
  "Complete Task" includes "Search Task" because the system must first locate the task before marking it as completed.
- **Extend Relationship:**
  "Delete Task" may extend from "Search Task" since a task must be identified before it can be removed.

Multi-Linked List Integration

The system internally uses a multi-linked list to manage tasks efficiently:

- **Subject List:** Groups tasks based on subject.
- **Priority List:** Organizes tasks according to importance.
- **Deadline List:** Sorts tasks based on due dates.

Each task node is connected across multiple lists, allowing efficient retrieval and updates without duplication.



12. TESTING AND VALIDATION – STUDENT STUDY PLANNER APPLICATION

Testing and validation are essential steps in software development to ensure that the system works correctly and meets user requirements. In the Student Study Planner Application, various testing methods were applied to verify the functionality, accuracy, and reliability of the system.

Testing Strategy

The application was tested using a combination of the following approaches:

- **Unit Testing:**
  Individual functions such as addTask(), deleteTask(), searchTask(), and completeTask() were tested separately to ensure they perform correctly.
- **Integration Testing:**
  Interaction between different modules (user input, controller, and data structure) was tested to verify smooth data flow.
- **System Testing:**
  The entire system was tested as a whole to ensure all features work together without errors.
- **User Acceptance Testing (UAT):**
  The system was tested by users (students) to confirm that it meets usability and functional requirements.

Sample Test Cases:

Test Case ID	Operation	Input Data	Expected Output	Result
TC01	Add Task	Valid task details	Task added successfully	Pass
TC02	Add Task	Empty fields	Error message displayed	Pass
TC03	Search Task	Valid Task ID	Task details displayed	Pass
TC04	Search Task	Invalid Task ID	"Task not found" message	Pass
TC05	Complete Task	Valid Task ID	Task marked as completed	Pass
TC06	Delete Task	Valid Task ID	Task deleted successfully	Pass
TC07	Display Tasks	Multiple tasks	All tasks displayed correctly	Pass

Validation Techniques

- **Manual Verification:**
  Each operation was manually checked to ensure correct output after execution.
- **Pointer Integrity Check:**
  Verified that all multi-linked list pointers (`nextBySubject`, `nextByPriority`, `nextByDeadline`) are correctly updated after insertion and deletion.
- **Data Consistency Check:**
  Ensured that each task node maintains consistent data across all linked lists.
- **Boundary Testing:**
  Tested edge cases such as empty list, maximum tasks, and invalid inputs.

13. RESULT AND DISCUSSION:

Results Summary

- The implementation of the Student Study Planner Application using a multi-linked list data structure produced efficient and reliable results in managing academic tasks. The system successfully enabled users to perform all core operations such as adding, searching, updating, and deleting tasks without any data inconsistency.
- The application demonstrated efficient task organization by allowing multiple views of the same data through subject-wise, priority-wise, and deadline-wise traversal. This multi-dimensional access significantly improved the speed and flexibility of retrieving task information compared to traditional single-linked list approaches.
- The system maintained consistent data updates across all linked lists. Whenever a task was modified, completed, or deleted, the changes were accurately reflected in all associated lists, ensuring data integrity. The application also handled a moderate number of tasks efficiently without noticeable performance degradation.
- Additionally, the progress tracking feature provided clear insights into completed and pending tasks, helping users monitor their study performance effectively.

Discussion:

- The Student Study Planner Application proves to be a practical and effective solution for managing study schedules in a structured manner. By utilizing a multi-linked list, the system overcomes common limitations of traditional task management methods.
- **Multi-Linked List Advantage:**
  The use of multiple pointers in each task node allows the system to maintain different views (subject, priority, deadline) without duplicating data. This reduces redundancy and improves retrieval efficiency.
- **Performance Efficiency:**
  The system ensures faster access to tasks based on specific criteria, minimizing the need for full traversal. This results in better performance compared to linear data structures.
- **Improved Time Management:**
  By clearly organizing tasks and tracking their completion status, the application helps students prioritize important work and meet deadlines effectively.
- **User-Friendly Operation:**
  The simple console-based interface makes the system easy to use, even for users with minimal technical knowledge.
- **Scalability and Flexibility:**
  The system can handle an increasing number of tasks and can be extended with additional features such as reminders, graphical interfaces, and database integration.
- **Error Handling and Reliability:**
  The application includes proper validation and pointer management techniques, ensuring no data loss or broken links during operations like deletion and updates.

14. LIMITATIONS

Despite its efficiency and usefulness, the Student Study Planner Application has certain limitations:

- **Memory Overhead:**
  The use of a multi-linked list requires multiple pointers for each task node, which increases memory consumption compared to simple data structures.
- **Complex Implementation:**
  Managing multiple pointers (subject, priority, deadline) makes the implementation more complex, especially during insertion and deletion operations.
- **Console-Based Interface:**
  The current system uses a basic console interface, which may not be visually appealing or user-friendly compared to modern graphical applications.
- **Limited Features:**
  The system does not include advanced features such as notifications, reminders, or synchronization with external devices.

- **Debugging Difficulty:**
  Errors related to pointer mismanagement can be difficult to trace and fix, especially in larger datasets.
- **Scalability Constraints:**
  While suitable for moderate data, handling extremely large datasets may require more advanced database systems.

15. CONCLUSION – STUDENT STUDY PLANNER APPLICATION

The Student Study Planner Application is an effective solution designed to assist students in organizing and managing their academic tasks efficiently. By utilizing a multi-linked list data structure, the system provides flexible and efficient task management through multiple views such as subject-wise, priority-wise, and deadline-wise organization.

The application successfully addresses common challenges faced by students, including poor time management, missed deadlines, and lack of proper planning. It enables users to add, update, delete, and track tasks in a structured manner, thereby improving productivity and academic performance.

One of the key strengths of this system is its ability to maintain data consistency while providing multiple traversal paths without redundancy. This demonstrates the practical application of advanced data structures in solving real-world problems.

Although the system has certain limitations, it provides a strong foundation for further enhancements. Future improvements such as graphical user interfaces, mobile integration, and automated reminders can make the application more powerful and user-friendly.

Overall, the project highlights the importance of structured planning and showcases how computer science concepts can be effectively used to develop meaningful and practical applications for everyday use.

16. REFERENCES

The development of the Student Study Planner Application was supported by a combination of academic textbooks, online resources, and programming documentation. These references provided essential knowledge on data structures, algorithms, and software development practices used in this project.

1. Reema Thareja, *"Data Structures Using C++"*, Oxford University Press – This book was used as a primary reference for understanding linked lists, memory management, and implementation techniques in C++.

2. Mark Allen Weiss, *"Data Structures and Algorithm Analysis in C++"*, Pearson Education – Provided in-depth knowledge on algorithm efficiency, data handling, and advanced data structure concepts.
3. Robert Lafore, *"Data Structures and Algorithms in C++"*, Sams Publishing – Helped in understanding practical applications of data structures and improving coding techniques.
4. GeeksforGeeks (https://www.geeksforgeeks.org) – A major online resource used for learning linked list operations, multi-linked list concepts, and solving programming problems.
5. W3Schools (https://www.w3schools.com/cpp/) – Used for basic C++ programming concepts, syntax, and examples for implementing functions.
6. TutorialsPoint (https://www.tutorialspoint.com) – Provided simple explanations and tutorials for understanding data structures and programming logic.
7. Stack Overflow (https://stackoverflow.com) – Used to resolve coding errors, debug issues, and understand real-world programming practices.
8. Lecture Notes and Classroom Materials – Provided foundational theoretical knowledge and guidance for the development of the project.
9. Online Articles and Technical Blogs – Various blogs and educational websites were referred to for understanding study planner concepts and improving system design.
10. YouTube Educational Channels – Video tutorials were used to understand practical implementation and visualization of linked list operations.
11. GitHub Repositories – Sample codes and project structures were referred to for better understanding of coding standards and project organization.
12. IEEE Papers and Research Articles – Referenced for understanding modern approaches to task management systems and software design principles.